

Git/Github 정리

1. Git의 시작

Git은 2005년 리누스 토발즈(Linux 개발자)가 개발
2002년 Linux 시스템의 버전 관리를 위해 Bit Keeper를 사용하려고 했으나 실패하고 개발
함.

Git이란, 소스의 변화를 계속 감시하고 있는 시스템

- 개발자의 코드 변경을 추적
- 변경 사항을 갱신

Git이 관리하는 공간을 저장소(Repository:repo)라고 하고, 저장소는 내 컴퓨터 환경(Local)안에 존재 => local 저장소의 코드 버전을 관리

로컬(local) repo : 내 컴퓨터에 저장된 프로젝트 & 코드

원격(remote) repo : Github 서버에 저장된 프로젝트 & 코드

[중요] 반드시 경로 내에 한글이 없어야 함.

작업은 로컬에서 -> 원격에 업로드

필요시 원격에서 로컬로 다운로드

Version Control System 방식

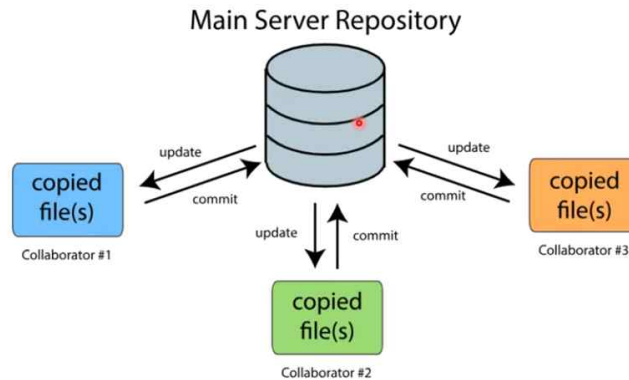


- 각각의 파일의 버전을 관리 해주는 것을 Version Control System이라고 한다.
- 사람이 직접 폴더명에 버전을 작성할 필요 없이 자동으로 해결

버전 관리 시스템은 두 가지 종류가 존재

1) Centralized Version Control(중앙 집중화 버전 관리 시스템 - 예전 방식)

=> CVS, ClearCase, Visual Source Safe, Team Foundation Server를 사용



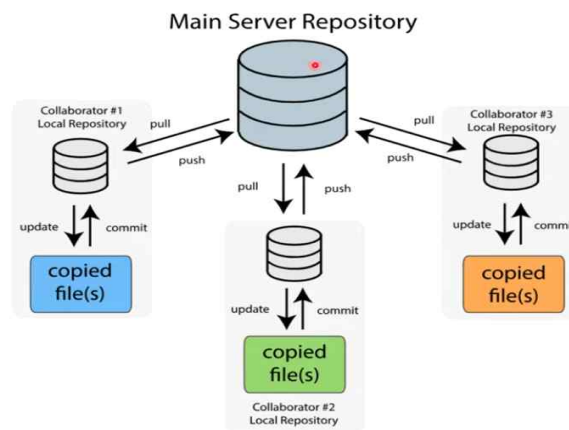
중앙 집중화 버전 관리 시스템은 2개의 방법을 사용

만약 Collaborator#1이 파일을 가져가서 수정 작업을 하면 중앙 저장소의 파일을 잠궈서 다른 사용자는 사용 or 편집 불가

또는 동시에 작업을 완료한 후 달라진 수정 부분을 관리자 또는 권한이 있는 사람이 확인하여 반영하는 방법을 사용

- 문제점 : 서버에 있는 가장 마지막 버전(최종 파일)이나 서버 자체가 깨지면 전체 프로젝트의 파일이 깨짐(원본이 1개)

2) Distributed Version Control(배포 되어있는 버전 관리 시스템 - 최신 사용 방식)



각 작업자가 메인 저장소의 풀 소스를 다운받고 Local Repository에서 각자 작업하는 방식
작업이 완료되면 중앙 메인 서버에 push. 이때 여러 사람이 같은 파일을 작업하고 push 할
경우 동시성 오류가 발생함. 이를 해결하기 위해 Branch(가지)를 각각 생성하여 합치는
과정을 진행, 원본이 깨지더라도 여러 Branch를 확인하여 원본 복구가 쉬움.

=> GNU arch, Git, Bit Keeper가 해당 방식이나 현재는 Git을 거의 사용함.

2. Git 설치

- Git 설치 공식 홈페이지(<https://git-scm.com>)에서 각각의 설치법을 확인(아래는 기본 설명으로 최신 버전마다 설치법 재확인하기)

1) Windows Version 설치시 주의할점

(1) 설치할 때, which editor would you like Git to use? 라는 항목에서 본인이 사용하고 싶은 에디터를 사용(기본 설정은 vim)

(2) How would you like to use Git from the command line? Use Git from Git Bash only는 선택하지 않기

=> 선택 시 윈도우 커맨드 프롬프트에서는 사용 불가

=> Git from the command line and also from 3rd-party software 선택

(3) 설치 완료 후 Git Bash 실행 or 윈도우 명령 프롬프트에서도 가능

=> test 코드 입력 (git --version)

2) Ubuntu Version

(1) 터미널 창 실행 후 설치 코드 입력(sudo apt-get install git)

=> test 코드 입력(git --version)

3) git 설치 완료 후 사용자 입력하기(github를 사용하기 위한 사용자 정보 입력)

=> --global 옵션은 개인으로 사용하는 곳에서만 사용(해당 명령어는 모든 저장소에서 한 번에 반영되게 하므로 팀 프로젝트나 공용 저장소가 있다면 영향을 미칠 수 있으니 주의해서 사용)

=> --global 옵션을 사용하지 않을 경우 로컬 저장소마다 입력해줘야함.

ex) --global 옵션 사용시

git config --global user.name "github 닉네임")

git config --global user.email "github 메일주소")

ex) --global 옵션 사용x

git config user.name "github 닉네임")

git config user.email "github 메일주소")

3. 저장소 만들기와 commit(커밋)하기-실습

Git 기본 명령어

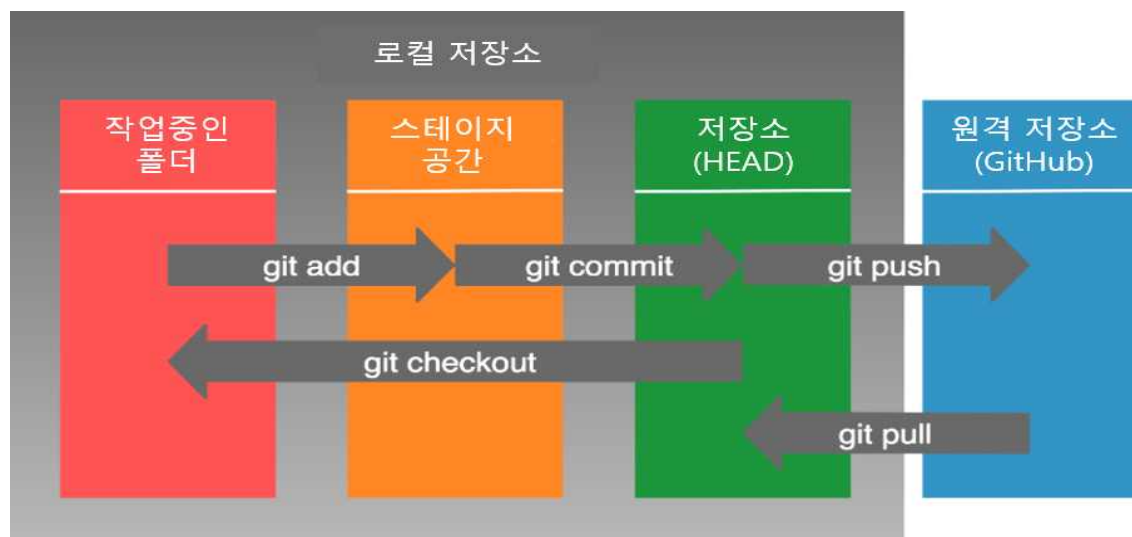
명령어	코드
저장소의 생성	git init
저장소에 파일 추가	git add <filename> -> ex) git add test1
저장소에 변경 내용의 반영	git commit, git commit m “메시지”
저장소의 상태 확인	git status
저장소의 기록 확인	git log

* Git add, commit, push란?

add : 내 Local에서 변경된 부분 중 저장소로 보낼 코드 선택 & 준비

commit : 준비 된 코드에 대한 설명 및 최종 확정

push : 업데이트 된 Local 저장소 내용을 Remote 저장소로 동기화



- 저장소 만들기 예시는 test_git 저장소에서 hello.html 파일을 이용하여 윈도우 환경에서 실습(윈도우 명령 프롬프트에서 실습)

1) 폴더 생성

(1) mkdir test_git

(2) cd test_git

(3) dir => 파일(디렉터리) 확인 명령어 - 아무것도 파일이 없는 것을 확인

(4) git init => 저장소 생성(Windows 환경에서는 히든 파일, 리눅스 or 맥에서는 보임)

2) 파일 생성

(1) notepad hello.html => hello.html 이름의 새로운 메모장 파일 생성

(2) 메모장에 아래 내용 입력 후 저장

```
<html>
<body>
  hello World
</body>
</html>
```

(3) dir => hello.html 파일 생성 확인

3) 생성된 파일의 상태 확인

(1) git status => 빨간색으로 표시되는 파일명 확인(파일이 변경되었음을 확인, 아직 저장소에는 추가 안 된 상태)

4) 저장소에 파일 추가

(1) git add hello.html => 정상적으로 추가되면 아무것도 안뜸

(2) git status => 파일이 초록색으로 표시되면 저장소에 추가 되었으나 확정된 상태는 아님
- ex) newfile : hello.html

5) 저장소에 파일 확정

(1) git commit

=> 에디터 창이 실행됨, 여기다가 메시지를 입력 후 저장. ex) Add hello.html

=> 해당 방식이 귀찮으므로 보통은 아래 명령어를 통해 사용

(1-1) git commit -m "Add hello.html"

(2) git status

=> 해당 명령어로 상태를 확인하면 클린하다고 표시됨.

6) 파일 재수정

(1) notepad hello.html

=> 메모장 내용 수정 (Hello World -> Hello World commit 2로 수정) 후 저장

7) git 상태 재확인 후 커밋까지

(1) git status

=> 수정된 파일이 빨간 글씨로 표시됨. ex) modified : hello.html

(2) git add hello.html

=> 다시 변경된 파일 저장소 추가

(2) git commit -m "Update hello.html"

=> 메시지 추가 후 파일 확정

8) 전체 과정 확인(log 확인)

(1) git log

=> 앞에서 hello.html 파일을 두 번 수정한 내용을 확인 가능, 이전 내용으로 돌아가기 위한 롤백 시간 확인 등이 가능

● Commit message

- 어떤 것을 변화 시켰는지를 파악하기 위해 알아보기 쉽게 적을 것(꼭 영어로 적을 필요 x)
- 특정 조직 마다 commit message 규칙이 존재(당장 적용할 규칙이 없다면, 일단 본인의 코드 변화를 잘 적는 연습 필요)
- commit의 제목과 본문 중 일단 제목만 잘 적기
- 간단한 메시지 추천
- 1) Add : ~ 기능을 하는 모듈/함수/파일 등을 추가함
- 2) Modify : ~기능 추가/업데이트 등을 위해 코드 변경
- 3) Fix : ~ 에러를 수정. 단순 typo / 논리 재구조화 / 입, 출력 변경

● reset : 스테이지에서 내리기

- add 명령어를 통해 수정된 파일을 스테이지에 올릴 수 있지만 잘못 add하여 스테이지에 올라갔을 경우 reset 명령어를 통해 스테이지에서 내릴 수 있다.

git reset <파일명>

해당 명령어를 사용하게 되면 작업한 내용은 변경되지 않고 단순히 스테이지에서만 뺄 수 있다. (git status 명령어를 사용해보면 상태가 변경된 것을 확인할 수 있음)

● .gitignore

- git이 추적하지 말아야 할 파일 목록(개인 설정 파일, 큰 데이터파일 등)을 정의하는 파일로 불필요한 파일이 원격 저장소에 올라가는 것을 방지하는 필수 기능

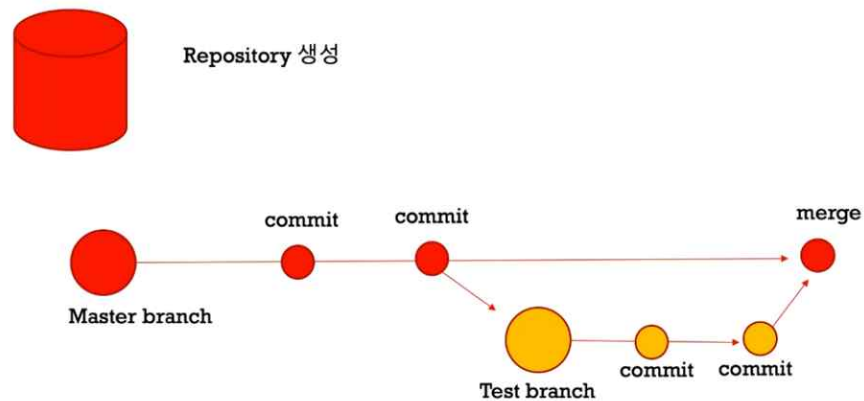
예시 : 프로젝트 폴더 안에 내 개인 비밀번호가 적힌 password.txt 파일이 존재할 경우 git add . 명령어나. git add* 명령어를 사용할 경우 password.txt 파일이 같이 올라감.

이를 방지하고자 gitignore파일을 생성하여 방지

1. notepad .gitignore -> .gitignore 파일명의 메모장 생성(이때 반드시 확장자명은 없어야함)
2. password.txt 작성 후 저장(제외하고 싶은 파일명을 확장자까지 작성)
3. 이제 git add . 이나 git add*을 해도 해당 파일이 올라가지 않음

4. Branch

- 생성된 저장소에서 별다른 지정을 하지 않았으면 Master Branch라는 공간에서 git 작업을 수행(commit)
- But, 동시에 여러명이 작업하는 경우는 Master Branch만으로는 부족, 새로운 Branch를 이용하여 새로운 작업 수행, 추후 관리자나 내가 새로운 Branch의 결과물이 마음에 들어서 Master Branch에 병합할 때는 Merge를 이용.



브랜치 명령어

명령어	코드
브랜치 확인	git branch
브랜치 생성	git branch <생성할 브랜치 이름>
브랜치 이동	git switch <브랜치 이름>
새로운 브랜치 만들면서 이동	git switch -c <새로운 브랜치 이름>
직전 브랜치로 돌아가기	git switch -

1) Branch 확인 명령어

(1) git Branch

=> 처음에는 *Master만 표시(*는 선택이 되어있다는 표시)

2) Branch 추가

(1) git branch “이름”

=> 실습에서는 test1으로 생성 -> git branch test1

3) Branch 변경

(1) git switch test1

(2) git branch

=> test1으로 변경됨 확인

4) 새로운 Branch에서 파일수정

(1) notepad hello.html

=> 실행된 hello.html 내용을 임의로 수정(수정해도 master에서는 확인 불가)

(2) git status

(3) git add hello.html

(4) git commit -m " Branch Test1"

5) 마스터 Branch 내용 확인

(1) git switch master => 마스터 branch로 이동

(2) notepad hello.html

=> test1 branch에서 수정한 내용이 반영 안 되었음.

6) 두 개의 branch를 병합

(1) git switch master

=> 항상 merge를 통해 병합하기 전 최종으로 병합해야 하는 곳으로 이동(test1 내용을 master에 반영하기 위해 master로 이동)

(2) git merge test1

=> test1 branch를 현재 branch(master)에 병합

(3) notepad hello.html

=> test1 branch에서 수정한 내용이 master branch에서도 반영됨.

7) branch 삭제

(1) git branch -d "이름"

=> git branch -d test1 (해당 명령어를 이용해 병합이 끝나 필요 없는 branch를 삭제)

5. restore

- 실수로 코드를 지웠거나, 수정한 내용을 되돌리고 싶을 때 등 '파일의 상태'를 복구할 때 사용
- 기존에는 Branch 관리 명령어인 checkout 명령어와 같이 사용하였지만 브랜치를 바꾸다 실수로 파일 내용을 덮어쓰거나 하는 혼란으로 2019년부터 swich와 restore로 기능 분리
- 해당 명령어는 작업 디렉토리의 모든 수정 사항을 완전히 삭제하여 되돌아가기 때문에 추후 복구가 불가능하니 사용시 주의하여 사용

명령어	코드
파일을 마지막 커밋 상태로 되돌림	git restore <파일명>
add 취소(파일 내용은 유지)	git restore --staged <파일명>
폴더 내 모든 수정 사항 취소	git restore .
특정 시점으로 복구	git restore --source=<해시><파일명>
선택적 복구 (수정된 부분 중 원하는 부분만 골라서 복구)	git restore -p <파일명>

1) 특정 시점 복구.

- git log --oneline(해당 명령어를 통해 복구하고 싶은 해시를 확인)

명령어를 입력하면 아래처럼 나옴, 여기서 앞의 7글자가 해시

a1b2c3d (HEAD -> main) 메인 페이지 UI 수정

f5e6d7c 로그인 기능 버그 해결

b9a8c7d 첫 커밋

-git restore --source=f5e6d7c main.py(해당 명령어를 입력하면 로그인 기능버그 해결시점으로 돌아감)

git stash

브랜치를 이동(switch)할 때 작업 중인 내용을 커밋하거나 되돌려야한다. 하지만 이때 커밋하거나 지우기 애매할 때 임시저장을 하는 방법.

예시 : main 브랜치에서 새 기능을 만들다가(50% 완성), 갑자기 다른 브랜치에 있는 내용을 급하게 수정해야할 때 현재 브랜치에서 작업 중인 내용을 임시저장하기 위해 사용

1. git stash -> 작업 중이던 코드가 사라지고 폴더가 깨끗해짐.
2. git switch <다른 브랜치> -> 이동 후 수정
3. git switch <원래 작업중이던 브랜치> -> 원래 작업중이던 브랜치로 다시 이동
4. git stash pop -> 사라졌던 코드가 다시 나타남.

6. Merge 충돌 해결

- 앞에서 생성한 branch들의 내용을 마스터나 다른 branch로 merge할 때 각각의 branch에서 파일이 수정되었을 경우 내용이 달라져 충돌 발생.
- 이때 해결 방법은 작업된 내용을 확인 후 병합한 다음 다시 add
- test_git 저장소를 새로 만들어 충돌 해결 실습

1) test_git 저장소(폴더 생성)

(1) mkdir test_git

(2) cd test_git

(3) git init

2) hello.html 파일 생성

(1) notepad hello.html => 메모장 안에 아래 내용을 입력 후 저장

<html>

<body>

</body>

</html>

3) 파일 상태 확인 및 추가

- (1) git status
- (2) git add hello.html
- (3) git commit -m "Add hello.html"

4) test Branch 생성

- (1) git branch Test_Branch => Test_Branch 생성
- (2) git branch => Test_Branch로 생성 확인
- (3) git checkout Test_Branch => master에서 Test_Branch로 이동
- (4) notepad hello.html => 아래 내용으로 수정

```
<html>
<body>
Edit in Test_Branch
</body>
</html>
```
- (5) git status => 파일 상태 확인(빨간색으로 파일이 수정된 것을 확인)
- (6) git add hello.html
- (7) git commit -m "Edit hello.html on Test Branch"

5) Master Branch에서 파일 수정

- (1) git switch master
- (2) notepad hello.html => master에서 hello.html 파일을 아래처럼 수정

```
<html>
<body>
Edit Master Branch
</body>
</html>
```

- (3) git add hello.html
 - (4) git commit -m "Edit hello.html on Master Branch"
- => 마스터 branch에서 파일 수정 후 커밋 완료

6) Master와 Test Branch의 내용이 동시에 수정됐을 경우 merge

- (1) git merge Test_Branch
- => Test_Branch 내용을 Master에 Merge 할 때 에러 발생
- => 에러코드 : Auto-merging hello.html
- CONFLICT(content) : merge conflict in hello.html
- Auto matic merge failed : fix conflicts and then commit
- => 이유 : 두 개의 Branch 모두에서 같은 파일이 수정되어 내용이 다를 때 발생

7. 원격 저장소 생성(Github)

- 1) github 접속 후 계정 생성
- 2) 로그인 후 my page에서 new(왼쪽 프로필 하단)을 클릭해서 Repositories 생성
- 3) Repositories name. Public(공개) or Private 여부 체크 후 Create
- 4) Initialize this repository with a README 체크(README 파일 생성은 필수)
- GitHub에서 README를 처음 생성하여 만들 경우 -> 로컬 저장소에서 반드시 Pull로 불러와야 됨.
- 5) 나머지는 전부 기본 상태 그대로 생성 완료

8. git과 github 연동에 필요한 패스워드 생성

- 1) 우측 상단에 본인의 프로필 이미지를 클릭
- 2) 목록 하단의 Settings를 클릭해서 뜨는 화면 왼쪽 리스트 하단에서 Developer Settings 클릭
- 3) 좌측의 Personal access tokens의 Tokens(classic)을 클릭
- 4) 우측 상단의 Generate new token을 클릭해서 new token(classic)을 클릭 후 github 패스워드 입력
- 5) Note에 토큰 이름을 입력, 하단의 Expiration은 본인이 해당 토큰을 사용할 기간을 입력(프로젝트마다 상이)
- 6) 원격 repo에 코드를 올리고 다운받기 위해서는 repo 항목을 반드시 체크 후 생성
- 7) 토큰이 생성되고 토큰 상단의 번호를 반드시 메모하거나 기억(추후 다시 확인 불가, 잃어버리면 토큰을 다시 생성해야 함).

9. 원격 저장소와 PR(Pull Request)

명령어	설명
clone	Remote repository의 내용을 복제
push	Remote repository로 보낸다
pull	Remote repository에 있는 내용을 가져온다
Fetch	Remote repository에서 변경된 내용 가져오기

1) Git 저장소와 GitHub와 연결하기

- (1) git remote add origin "GitHub 저장소 주소"
-> 여기서 origin은 git에서 관례적으로 사용하는 원격 저장소의 별명
- (2) git branch -M main -> GitHub에서 저장소를 생성할 경우 기본 branch가 main으로 생성되므로 로컬 저장소도 master에서 main으로 변경
(2-1) git init -b main 명령어로 처음부터 로컬 저장소를 생성할 때 branch를 main으로 생성

2) 내 저장소 주소 확인하기

현재 어떤 GitHub 저장소와 로컬 저장소가 연결되어 있는지 확인하는 명령어

(1) `git remote -v` 명령어로 확인

만약 연결된 주소가 틀리다면 `git remote remove origin` 명령어로 지우고 다시 연결하기

-> 다시 연결 `git remote add origin "GitHub 저장소 주소"`

3) Git 저장소 내용을 GitHub로 전송

(1) `git add`와 `commit` 명령어로 수정된 내용 업데이트

(2) `git push origin main`

=> 로컬 저장소에만 `commit`된 내용을 github에도 반영(git에 사용자 정보가 입력되어 있어야 함)

=> 처음 입력할 경우 github로그인 or 토큰입력창 나옴.

3-1) `Push -u` 옵션 사용

새로 생성한 커밋들을 원격 저장소에 업로드 할 때 사용. 기존 `push` 명령어에 `-u`옵션을 사용하면 해당 브랜치의 업스트림을 등록할 수 있음. 이때 한번 등록하면 `git push` 명령만으로 원격 저장소에 업로드 됨.

-> 여기서 업스트림 지정은 로컬 저장소와 연결된 원격 저장소를 일컫는 단어로 설정해두면 `git push origin main` 과 같이 저장소와 브랜치를 지정하지 않아도 `git push` 명령어로만 `push`가 가능.

4) Github에 있는 저장소(Repositories) 다운

(1) `cd ..`

(2) `cd "본인이 연결한 파일 경로"` => 로컬 저장소에서 작업 시작

(3) `git clone "github 주소"`

=> github에 생성한 저장소가 다운됨.

=> 파일 등을 수정하고 `git status` 명령을 통해 확인

(4) `git status`

(5) `git add *`

=> `add` 뒤에 `*`를 붙이면 변동된 모든 파일 추가(이때 저장소에 있는 모든 파일을 올리게 되는데 임시 파일이나 올리면 안되는 파일도 같이 올라가기 때문에 주의, `gitignore` 파일 생성법을 참고해서 사용)

(6) `git commit -m "메시지 내용"`

5) Fetch vs Pull의 차이

- 원격 저장소(GitHub)의 최신 코드를 로컬로 가져오는 방식의 차이

예시 : 팀원이 GitHub에 새 코드를 올렸다.

1. Fetch(보고만 오기) : `git fetch origin`

- 어떤 것이 바뀌었는지 확인만 할 때 사용.

- 내 로컬 저장소의 내용은 변하지 않고 원격의 업데이트 내역만 확인할 때 사용.(로그로 확인 후 합쳐도 되는지 아니면 안되는지 본 다음 병합)

- `git log --oneline main..origin/main` 명령어로 새로운 커밋이 있는지 확인.
- `git diff main origin/main` 명령어로 어떤 코드 줄이 수정되었는지 비교
- `git merge origin/main` 명령어를 통해 병합 or 파일 직접 수정

2. Pull(합치기) : `git pull origin main`

- 확인이고 뭐고 일단 내 파일과 합칠 때 사용
- 팀원이 새로 올린 코드로 즉시 업데이트(merge)
- 만약 위 코드로 오류가 발생한다면 원격 저장소 내용 강제로 가져오기
`git pull origin main --allow-unrelated-histories`

결론 `git pull = fetch + merge`

추가 명령어

1. 원격 저장소 브랜치 삭제하기(예시 master 브랜치)

`git push origin --delete master`

2. 로컬에서 브랜치 삭제하기(예시 master 브랜치)

`git branch -d master`

3. 원격 저장소 삭제하기

1. GitHub 원격 저장소 삭제하기 (웹사이트에서)

GitHub 사이트에 올라간 저장소 자체를 없애고 싶을 때 사용합니다. 이 작업은 되돌릴 수 없으니 주의하세요!

1. 삭제하려는 **GitHub 저장소(Repository)** 페이지로 들어갑니다.
2. 상단 메뉴의 **Settings** (톱니바퀴 아이콘)를 클릭합니다.
3. 가장 아래쪽으로 스크롤하여 **Danger Zone** 섹션을 찾습니다.
4. **Delete this repository** 버튼을 클릭합니다.
5. 확인을 위해 저장소 이름(예: `Jsm9902/test2`)을 직접 입력하라는 창이 뜨면 그대로 입력합니다.
6. 최종 삭제 버튼을 누르면 완전히 삭제됩니다.

4. 로컬 저장소 연결 해제

- 터미널/명령 프롬프트(CMD)에서:

```
Bash
# 해당 프로젝트 폴더로 이동한 상태에서
rm -rf .git
```

(Windows CMD의 경우: `rmdir /s /q .git`)

5. 로컬 저장소 완전 삭제

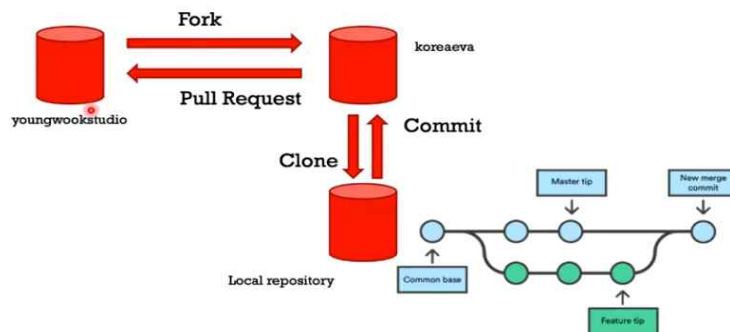
내 컴퓨터에서 프로젝트 폴더 자체를 완전히 없애고 싶다면, 윈도우 탐색기에서 마우스 오른쪽 버튼을 눌러 삭제를 누르거나 터미널에서 다음을 입력합니다.

```
Bash
cd ..
rm -rf newtest # 'newtest' 폴더 전체를 삭제
```

Windows CMD의 경우 rmdir -rf newtest

9. PR(Pull Request)가 매우 중요한 요소

- 다른 사람의 github 저장소의 내용을 내가 수정하고 싶을 때(혹은 다른 계정의 저장소의 내용을 내가 수정하고 싶을 때) Fork 기능과 RP를 사용



Fork는 다른 사람의 Repository에 있는 내용 전체를 내 Repository로 옮겨서 작업 가능
=> 이렇게 하는 이유는 다른 계정의 Repository의 내용을 수정하고 싶을 때 권한이 없는 부분 때문이다.

- (1) 수정 or 변경하고 싶은 다른 계정의 Repository를 내 계정으로 Fork
- (2) 내 Github 계정에 Fork 한 Repository를 Local Repository로 clone
- (3) 파일 수정(Branch, add, commit, merge 등)을 한 뒤 내 github로 수정된 Local Repository 내용을 push
- (4) 내 github에서 최종 검수 후 다시 가져왔던 다른 계정으로 Pull Request
- (5) 다른 계정의 Repository 관리자가 확인 후 merge 승인 or 리젝

10. VScode에서 git 사용

1) 작업 공간에 코드 추가

=> VScode에서 생성한 local git 저장소(폴더)를 open

2) VScode 탐색기에 추가된 저장소 내용에 파일을 생성

=> ex) Upload_test.py

3) 생성 후 git 명령어를 사용하여 add, commit, push 등을 수행

4) VScode에서 바로 사용하는 방법은 왼쪽 목록바에서 아래의 아이콘을 클릭

=> Git 관련 컨트롤을 해줄 수 있는 기능을 제공



5) 파일명 옆에 +버튼을 누르면 add가 됨(add를 잘못된 경우 -를 눌러 다시 취소)

6) commit은 해당 파일을 선택하고 파일 리스트 상단 채팅방에 메시지를 입력하고 커밋 버튼을 눌러 commit

7) commit을 완료하고 push 할 때는 소스 제어 옆 ...을 눌러서 리스트 중에 push를 누르면 완료